

AI 웹 개발: 내 아이디어를 현실로, AI 웹 서비스 빌딩

TripMate AI : AI가 만들어주는 나만의 여행서비스 최종 결과 보고서

https://github.com/bzkhseo-source / TripMate-AI_C

배포 URL: <https://tripmate-ai-five.vercel.app>

1. 미션 개요

AI 기반 여행지 추천 웹서비스 'TripMate AI'를 기획부터 배포까지 전 과정을 직접 수행하는 미션.

여행 날짜와 지역을 입력하면 AI가 여행지를 추천하고, 관련 행사·숙박·맛집·영상 정보를 통합 제공하며, 여행 계획을 저장하고 관리할 수 있는 기능까지 구현하는 것을 목표.

HTML/CSS/JS 프론트엔드와 Python(Flask) 백엔드로 구성하고, Google Gemini API, Google Places API(New), 한국관광공사 TourAPI, Ticketmaster Discovery API, YouTube Data API v3를 연동하였으며, 최종적으로 Vercel Serverless Function 구조로 배포.

2. 과제 목표 달성 체크리스트

- ☒ 여행 날짜/지역 입력 → AI 여행지 추천 기능 구현
- ☒ Google Places API 기반 숙박/맛집 정보 제공 (평점, 리뷰수 포함)
- ☒ 여행 기간 내 국내/해외 행사 정보 제공
- ☒ 인기 여행 영상 TOP 3 제공
- ☒ 보너스 과제: 여행 계획 저장 및 방문상태 관리 기능 구현
- ☒ API 키 보안 관리 (.env, .gitignore, .env.example)
- ☒ 예외 처리 (빈 입력, 0건 결과, 인증 실패, 쿼터 초과 등)
- ☒ GitHub 공개 저장소 커밋 이력 관리
- ☒ Vercel 실 배포 및 정상 동작 확인
- ☒ 데스크톱/모바일 반응형 검증

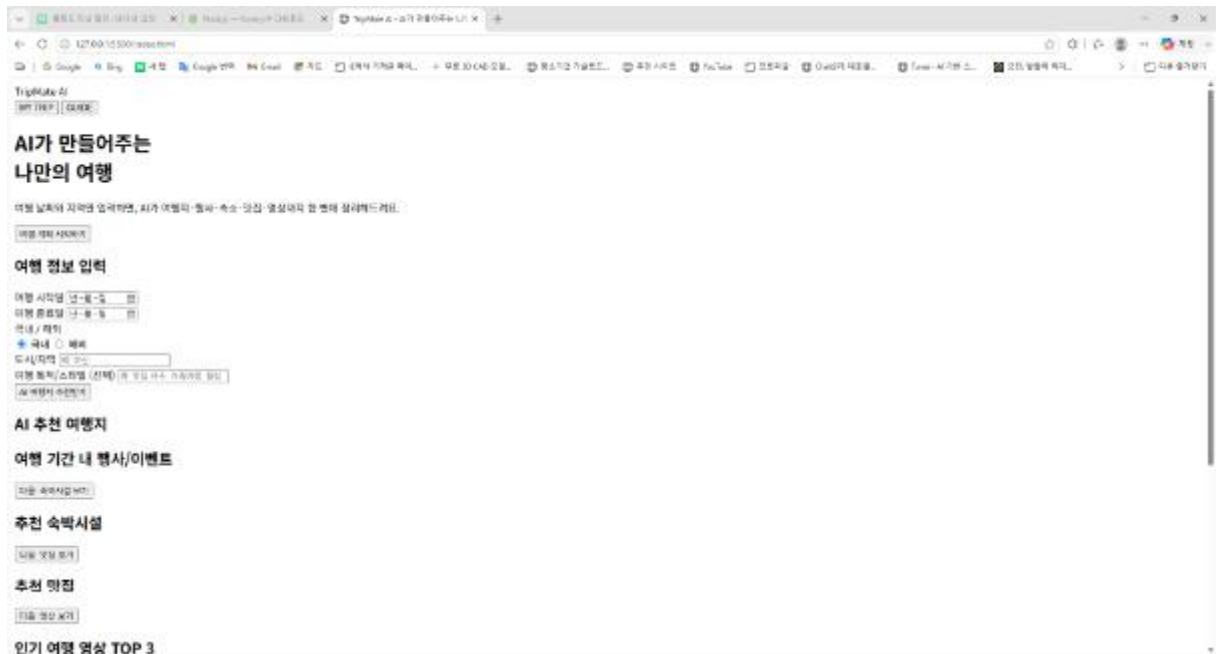
3. 기능 요구사항 이행 현황

요구 내용	구현 위치	상태
여행 날짜/지역 입력 폼	index.html, js/app.js (setupTripForm)	완료
국내/해외 계층 선택 UI	js/app.js (setupRegionToggle)	완료
AI 여행지 추천 (10곳 이상)	server/recommend.py	완료
행사 정보 (국내 TourAPI)	server/events.py	완료
행사 정보 (해외 Ticketmaster)	server/events.py	완료
숙박시설 추천 (평점 기반, 10곳 ↑)	server/places.py	완료
맛집 추천 (평점 기반, 10곳 ↑)	server/places.py	완료
인기 영상 TOP 3	server/youtube.py	완료
여행 계획 저장 (보너스)	js/app.js (saveCurrentTrip 등)	완료
방문상태 관리 (보너스)	js/app.js (toggleTripVisited)	완료
저장 계획 상세보기 (보너스)	js/app.js (loadSavedTrip)	완료
API 키 보안 (.env)	.gitignore, .env.example	완료
예외처리 (빈 입력/0건/인증오류)	server/*.py 전역	완료
24시간 캐싱	server/cache.py	완료
Vercel 배포	api/index.py, vercel.json	완료

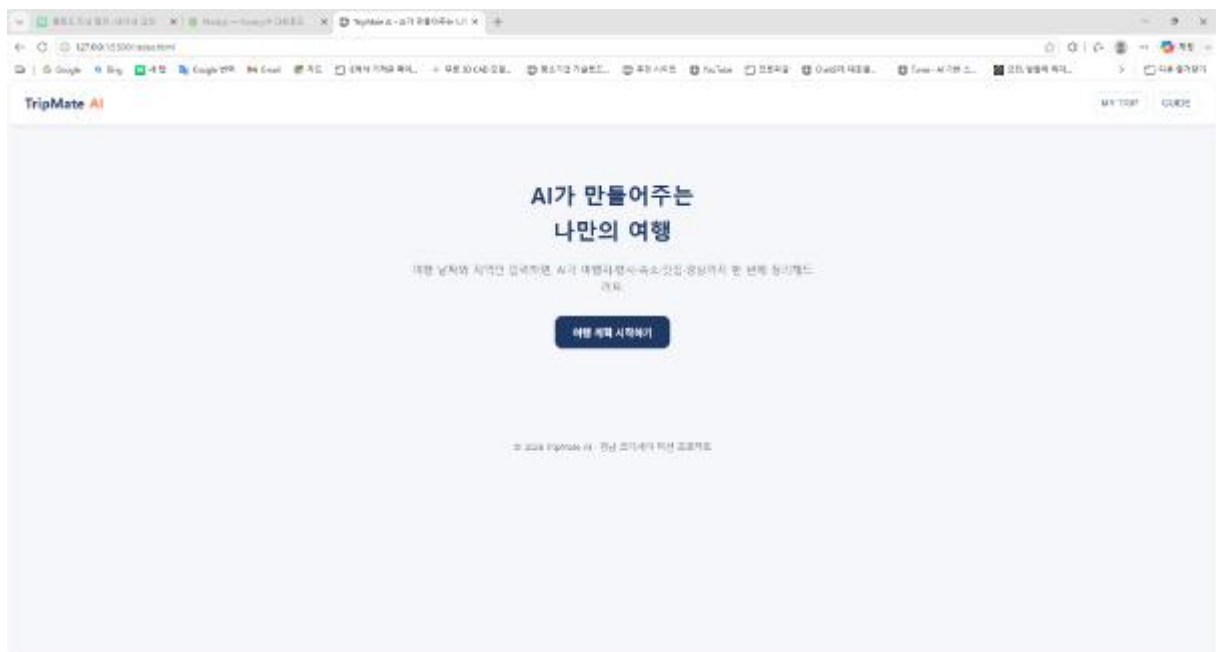
4. 결과물 상세

4-1. 화면 UI 구현 (HTML/CSS)

남색(#1F3864)과 주황색(#FF7A45) 포인트 컬러로 신뢰감 있는 톤을 구성하였고, 데스크톱과 모바일 모두에서 자연스럽게 렌더링되도록 반응형 CSS를 적용.



HOME 화면 - 초기 HTML 구조 렌더링



CSS 적용 후 데스크톱 HOME 화면

4-2. 입력 검증 및 화면 전환 (JavaScript)

빈 입력 상태로 제출 시 브라우저 기본 검증 대신 커스텀 오류 메시지가 표시되도록 구현.

빈 입력 시 커스텀 오류 메시지 표시

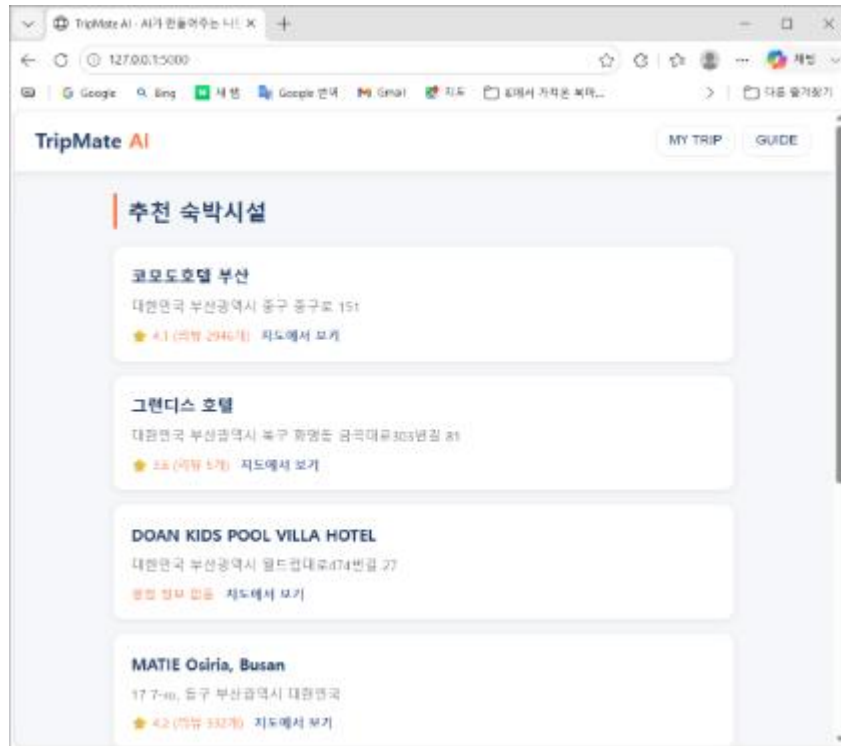
4-3. 결과물 1 - AI 여행지 추천

Gemini API(gemini-3.5-flash)를 구조화된 JSON 스키마(Pydantic)로 호출하여, 실제 존재하는 여행지를 추천 이유·예상 체류시간·일정 적합성과 함께 최소 10곳 이상 제공.

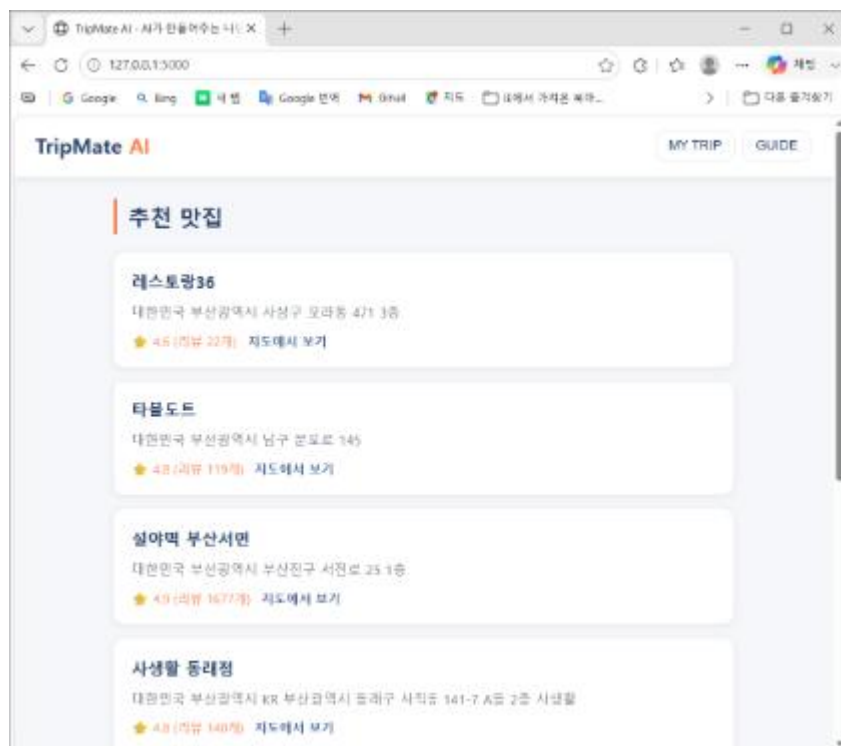
부산 여행 AI 추천 결과 (해운대 블루라인파크 등)

4-4. 결과물 2 - 숙박시설 및 맛집 정보

Google Places API(New)의 Text Search를 통해 평점과 리뷰수를 포함한 숙박시설·맛집 정보를 각 10곳 이상 제공하며, 평점 정보가 없는 업체도 안전하게 처리.



부산 숙박시설 추천 결과



부산 맛집 추천 결과

4-5. 결과물 3 - 행사/축제 정보

국내는 한국관광공사 TourAPI(KorService2 searchFestival2), 해외는 Ticketmaster Discovery API를 사용하여 여행 기간 내 행사 정보를 제공.



부산 지역 행사 정보 (광안리 드론쇼, 인디컱트페스티벌 등)

4-6. 결과물 4 - 인기 여행 영상 TOP 3

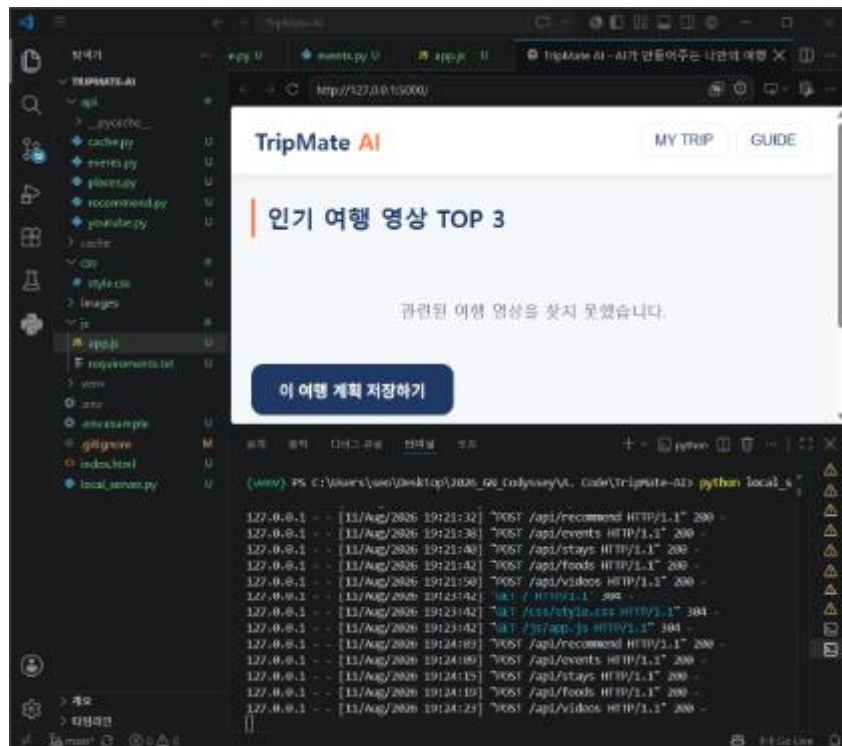
YouTube Data API v3로 추천 맛집 이름 기반 검색을 우선 시도하고, 결과가 없으면 지역 맛집 일반 검색어로 자동 재시도하는 다단계 검색 로직을 구현하여 검색 성공률을 높임.



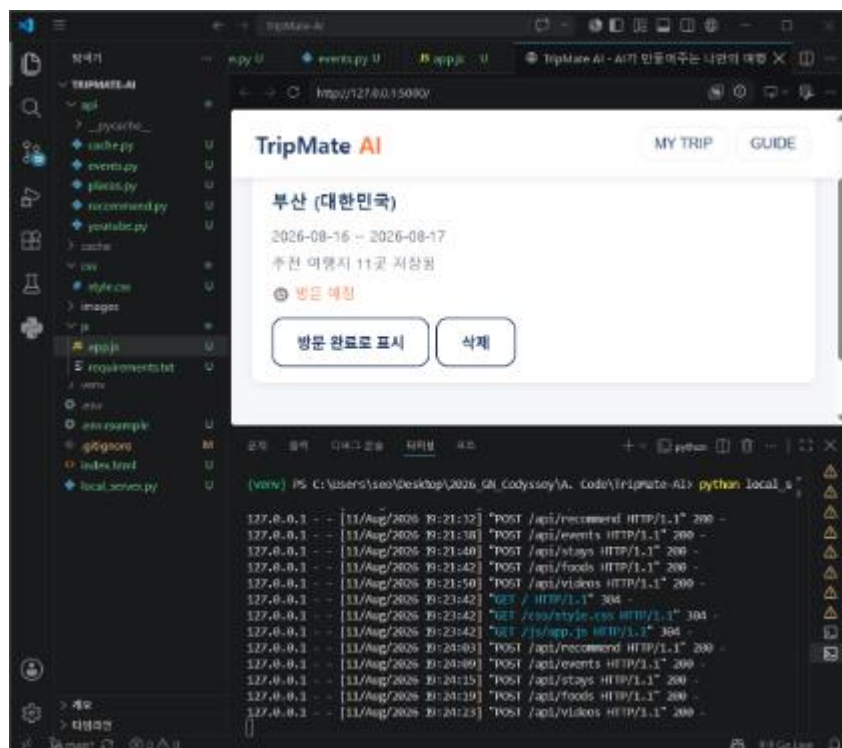
검색어 다양화 개선 후 실제 관련 영상 노출 결과

4-7. 결과물 5 - 여행 계획 저장 기능 (보너스)

여행 계획 저장 시 브라우저 localStorage에 AI추천.행사.숙박.맛집.영상 정보 전체를 저장하며, MY TRIP 화면에서 목록 확인, 방문상태(방문예정/방문완료) 토글, 삭제, 그리고 재조회 없이 즉시 확인 가능한 상세보기 기능을 제공.



여행 계획 저장 완료 알림



MY TRIP 화면 - 저장된 여행 계획 목록 및 방문상태 표시

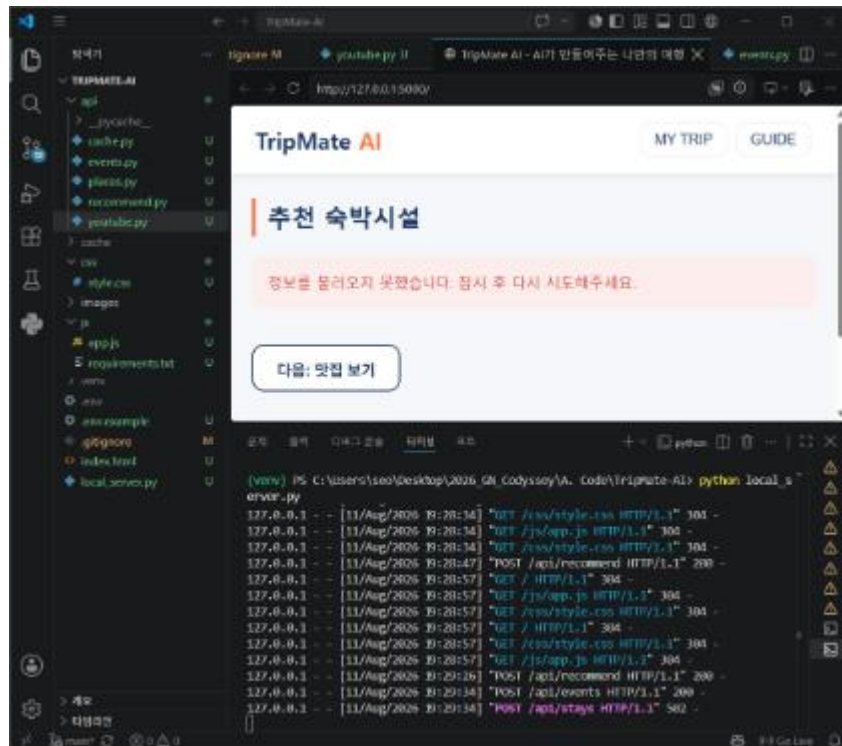
5. 보너스 과제 구현 내역

보너스 과제로 '여행 계획 저장 기능'을 선택하여 구현하여 단순 저장을 넘어, 다음 세 가지를 추가로 구현함.

- 방문상태 관리: 저장된 각 여행에 방문예정/방문완료 상태를 부여하고 토글 가능하도록 구현
- 상세보기(캐시 활용 UX): 저장된 여행을 다시 볼 때 API를 재호출하지 않고 저장된 데이터를 즉시 렌더링하는 viewOnly 모드를 설계하여, 불필요한 API 낭비를 막으면서도 빠른 UX를 제공
- localStorage 기반 클라이언트 저장: 별도 로그인/DB 없이도 개인화된 저장 기능을 구현할 수 있음

6. 트러블슈팅 기록

오류 상황	원인	해결 과정
venv 활성화 스크립트 미인식	python -m venv venv 명령이 정상 실행되지 않았거나 다른 폴더에서 실행	pwd/dir로 폴더 위치 확인 후 venv 재생성
npm 인식 불가	Node.js 미설치	nodejs.org에서 LTS 설치 후 VS Code 재시작
Gemini 모델명 오류 (gemini-2.5-flash)	구형 모델명 사용, 실제로는 gemini-3.5-flash로 변경됨	web_search로 최신 모델명 확인 후 수정
TourAPI 403 SERVICE_KEY_IS_NOT_REGISTERD_ERROR	Encoding 키를 requests 파라미터로 전달 시 이중 인코딩 발생	Decoding 키로 교체하여 해결
TourAPI keyword 파라미터 오류	searchFestival2는 keyword 파라미터를 지원하지 않음	keyword 제거, 전국 조회 후 도시명으로 클라이언트 필터링
YouTube 검색 결과 0건	특정 맛집 이름 기반 단일 검색어의 낮은 적중률	맛집 이름 기반 + 일반 검색어 등 다중 검색어 순차 시도로 개선
Gemini 무료 티어 요청 제한 도달	반복 테스트로 분당/일일 쿼터 소진	24시간 파일 캐시(server/cache.py) 도입
Git detached HEAD 상태로 push 실패	rebase 중단 상태에서 명령 충돌	rebase --abort 대신 새 GitHub 저장소로 안전하게 재구성
Vercel 배포 후 404 Not Found	vercel.json의 rewrites 방식이 Flask WSGI 요청을 온전히 전달하지 못함	builds+routes 방식으로 vercel.json 교체
Vercel 프로젝트 생성 오류	프로젝트 이름에 대문자 포함	소문자 프로젝트명으로 재생성

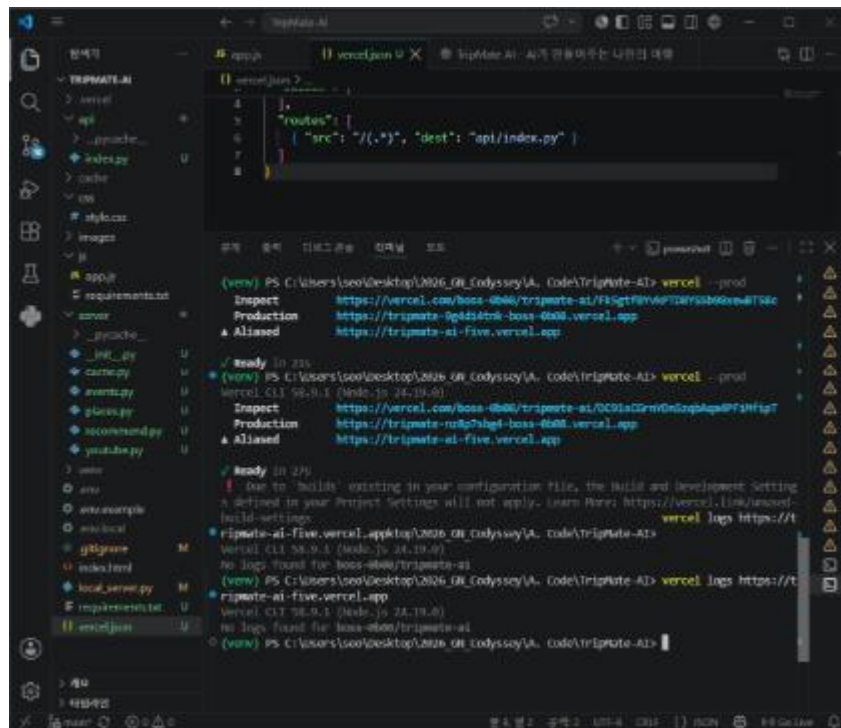


예외처리 검증: 잘못된 키로 발생시킨 오류가 화면 크래시 없이 안전하게 표시됨

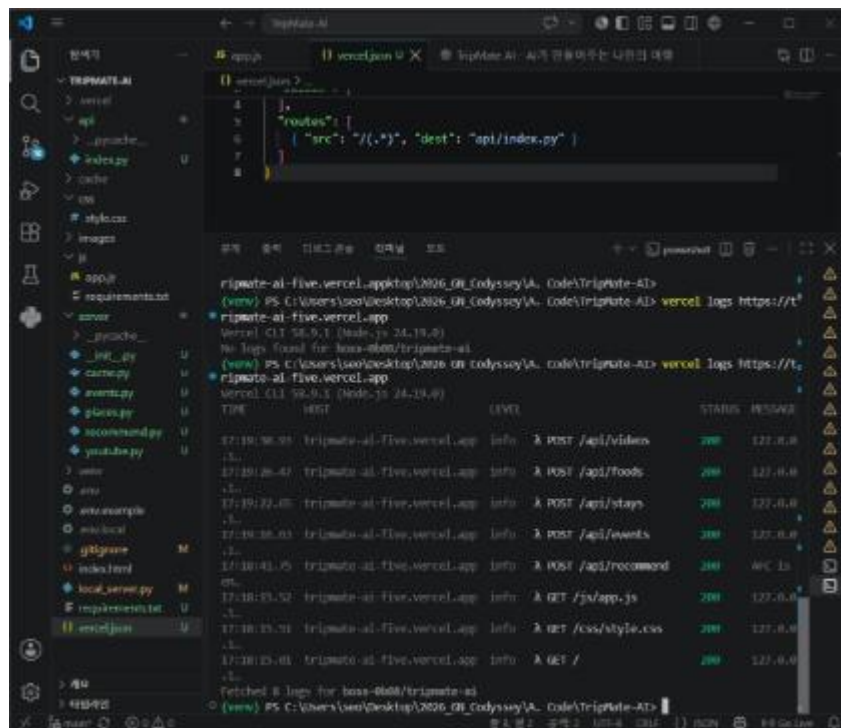


키 복구 후 정상 데이터 재확인

7. 배포 검증



로컬 검증 완료 후 Vercel 배포 성공 로그



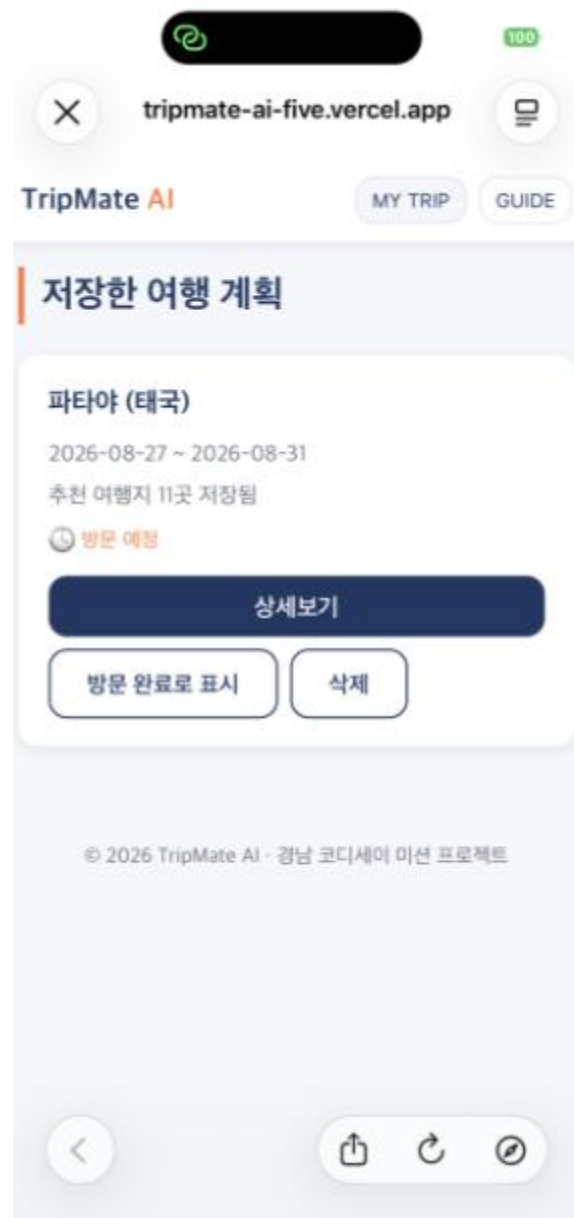
프로덕션 URL 실제 요청 로그 - 전 API 200 정상 응답



배포된 사이트 데스크톱 HOME 화면



배포된 사이트 모바일(실기기) HOME 화면



배포된 사이트 모바일 MY TRIP 화면

8. 최종 결론

이번 미션을 통해 프론트엔드(HTML/CSS/JS)부터 백엔드(Python/Flask), 5개의 서로 다른 외부 API 연동, 그리고 실제 서버리스 플랫폼(Vercel) 배포까지 웹 서비스 개발의 전체 주기를 실행.

아래와 같은 실무적인 문제 파악.

- 공식 문서와 실제 API 응답을 근거로 문제를 진단 (모델명 변경, 파라미터 오류, 인증키 종류 문제 등)
- 파일 단위 전체 교체가 부분 수정(찾기-교체)보다 안전한 상황을 판단
- API 키와 같은 민감 정보를 다루는 보안 (.env, .gitignore, 노출 시 즉시 재발급)
- 로컬 개발 구조와 서버리스 배포 구조의 차이를 이해하고 재구성 (Flask 모듈 구조 → Vercel 서버리스 함수 구조)
- 무료 티어 쿼터 등 실제 서비스 운영에서 마주치는 제약을 캐싱으로 완화

완성된 TripMate AI는 여행 날짜와 지역 입력만으로 AI 추천부터 행사·숙박·맛집·영상 정보, 그리고 개인화된 여행 계획 저장까지 지원하는 완결된 웹 서비스로, 데스크톱과 모바일 환경 모두에서 정상 동작함을 확인하였음.

9. 미션 요구사항

항목	상태
실제 배포 웹서비스 (Vercel)	완료 - https://tripmate-ai-five.vercel.app
GitHub 공개 저장소	완료 - github.com/bzkhseo-source/TripMate-AI_C
README.md	완료
서비스 기획서	완료
증빙자료 (스크린샷)	완료 - 본 보고서에 포함
AI 여행지 추천 기능	완료
행사 정보 (국내+해외)	완료
숙박/맛집 정보 (평점 기반)	완료
인기 영상 TOP 3	완료
보너스: 여행 계획 저장	완료
API 키 보안 관리	완료
예외처리 (전 시나리오)	완료
데스크톱/모바일 반응형	완료

10. 체크리스트

평가 항목	충족 여부
배포 URL 접속 가능 + 3개 이상 페이지 섹션이 메뉴 이동으로 확인	충족 - HOME, TRIP INPUT, AI RECOMMEND, EVENTS, STAY, FOOD, YOUTUBE, MY TRIP, GUIDE 총 9개 화면, 상단 MY TRIP과 GUIDE 메뉴 및 단계별 이동 버튼으로 확인 가능
모바일 반응형 정상 동작	충족 - 실기기 테스트로 레이아웃 깨짐, 가로 스크롤, 버튼 잘림 없음을 확인
AI 기능 입력에서 요청, 결과 표시까지 정상 동작	충족 - TRIP INPUT 제출 시 recommend API 호출, 로딩 표시 후 결과 카드 렌더링까지 확인
정상, 빈값, 긴 입력 등 테스트 재현 가능	충족 - 정상 입력, 빈 입력(날짜 미입력), 잘못된 인증키 입력 등 다수 시나리오를 실제로 재현하고 결과를 기록
실패 상황에서 사용자 안내 메시지 1개 이상 표시	충족 - 빈 입력, 0건 결과, 인증 오류, 쿼터 초과 각각에 대해 서로 다른 안내 문구 구현
API 키가 코드에 노출되지 않고 환경 변수로만 관리	충족 - .env, .env.example, .gitignore로 관리, Vercel Environment Variables에 등록. 개발 중 1회 노출 사고 발생 시 즉시 재발급 조치
제출 패키지(필수 5종) 제공	충족 - 배포 URL, GitHub 저장소, README.md, 서비스 기획서, 본 최종 보고서(증빙자료 포함)

11. 설계 참고 사항

11.1 html, css, js, api 구조를 이렇게 나눈 이유

역할별로 파일을 분리하여 각 파일이 하나의 책임만 갖도록 설계.

- index.html은 화면 구조(9개 section)만 담당
- css/style.css는 시각적 스타일과 반응형 규칙만 담당
- js/app.js는 화면 전환과 입력 검증, API 호출, 상태 관리(tripState)를 담당.
- api/index.py는 Vercel이 인식하는 유일한 서버리스 진입점이고, 실제 비즈니스 로직(AI 추천, 장소 검색, 행사 조회, 영상 검색, 캐싱)은 server 폴더 아래 기능별 모듈로 분리.

이렇게 분리한 이유는, 한 파일이 너무 커지면 수정 시 의도치 않은 부분이 함께 깨지는 문제를 실제로 여러 차례 겪었기 때문이며, 기능 단위로 파일을 나누면 특정 API를 수정할 때 다른 기능에 영향을 주지 않고 독립적으로 테스트할 수 있음.

11.2 AI 기능 프론트 로직의 로딩, 성공, 실패 상태 처리

js/app.js의 fetchAiRecommendations 함수를 기준으로,

- 요청 시작 시 로딩 박스를 표시하고 에러 박스와 결과 목록을 초기화.
- fetch 응답이 정상이면 데이터를 tripState에 저장하고 결과 카드를 그린 뒤 다음 버튼을 노출.
- 응답이 실패하거나 네트워크 예외가 발생하면 서버가 반환한 오류 메시지를 에러 박스에 표시.
- 성공과 실패 여부와 무관하게 마지막에는 로딩 표시를 종료.

- 이 패턴을 행사, 숙박, 맛집, 영상 조회에도 동일하게 재사용.

11.3 Serverless Function(Python)의 입력 검증과 응답 포맷

- `api/index.py`의 각 라우트는 공통적으로 요청 본문을 안전하게 파싱한 뒤, 필수 필드(여행 시작 일, 종료일, 도시)가 비어있으면 즉시 400 상태 코드와 오류 메시지를 담은 JSON을 반환.
- 정상 처리 시에는 200과 함께 결과 데이터를, 외부 API 오류로 인한 실패 시에는 각 모듈에서 정의한 커스텀 예외를 잡아 502로, 그 외 예상치 못한 예외는 500으로 응답하되 항상 동일한 오류 메시지 포맷을 유지.
- 이렇게 응답 포맷을 통일한 이유는 프론트엔드가 성공, 실패 여부와 무관하게 동일한 파싱 로직으로 처리할 수 있도록 하기 위함.

11.4 배포 후 문제 진단 순서 (로그, 콘솔, 재배포)

실제로 배포 직후 페이지를 찾을 수 없다는 오류가 발생했을 때 다음 순서로 진단

- 첫째, 브라우저에 표시된 오류 메시지의 형식을 먼저 확인하여 애플리케이션 자체 오류인지 플랫폼 오류인지 구분
- 둘째, Vercel 로그 명령으로 실제 서버리스 함수에 도달한 요청과 상태 코드를 확인
- 셋째, 배포 설정 파일의 라우팅 방식이 Python 웹 함수에 요청을 온전히 전달하는 방식인지 공식 문서와 대조
- 넷째, 설정을 검증된 방식으로 교체한 뒤 다시 배포
- 다섯째, 재배포 후 다시 로그로 실제 정상 응답을 확인하여 해결 검증.

11.5 HTML, CSS, JavaScript의 역할 차이 (코드 근거)

- `index.html`은 의미 있는 구조와 데이터가 들어갈 빈 컨테이너만 정의
- `css/style.css`는 화면 요소의 표시 여부와 활성 화면 스타일처럼 어떻게 보일지만 정의
- `js/app.js`는 `showScreen` 함수로 클래스를 토글하여 언제 어떤 화면을 보여줄지를 결정하고, `fetch`로 받아온 데이터를 바탕으로 DOM 요소를 동적으로 생성.
- 즉 HTML은 구조, CSS는 표현, JS는 동작과 데이터 바인딩이라는 역할 분리가 프로젝트 전반에 일관되게 적용.

11.6 fetch 요청이 서버리스 함수로 전달되고 응답이 돌아오는 흐름

- 사용자가 폼을 제출하면 `js/app.js`에서 POST 방식으로 추천 API를 호출.
- 이 요청은 `vercel.json`의 라우팅 설정에 의해 Vercel 플랫폼이 `api/index.py`에서 정의된 Flask 앱으로 전달

- Flask는 요청 경로와 메서드에 매칭되는 라우트 함수를 실행하고, 그 안에서 server/recommend.py의 함수를 호출해 Gemini API와 통신
- 결과는 JSON 응답으로 변환되어 다시 Vercel을 거쳐 브라우저로 돌아오고, 프론트의 비동기 파싱 로직이 이를 처리하여 화면에 반영.

11.7 환경 변수를 사용하는 이유 (보안, 운영)

- API 키를 코드에 직접 작성하면 GitHub에 업로드되는 즉시 누구나 볼 수 있어 악용될 위험.
- 환경 변수로 분리하면 코드 저장소와 비밀 값이 물리적으로 분리되어, 저장소가 공개되어도 키는 노출되지 않음.
- 또한 로컬과 배포 환경에서 서로 다른 값을 사용할 수 있어 운영 유연성도 확보.

11.8. AI 기능을 넣은 이유와 프롬프트 구성 방법

- 여행지 추천은 단순 검색으로는 사용자의 여행 기간과 스타일을 종합적으로 고려하기 어려운 영역이라 AI를 도입
- server/recommend.py의 프롬프트 생성 함수는 여행 기간, 국가, 도시, 여행 스타일을 명시적으로 프롬프트에 삽입하고, 실제로 존재하는 장소만 추천할 것, 최소 10개에서 최대 12개, 체류시간과 일정 적합성 포함, 단순 나열이 아닌 동선을 고려한 추천이라는 네 가지 제약을 명시.
- 또한 구조화된 응답 스키마를 지정하여 AI가 항상 정해진 JSON 구조로만 응답하도록 강제함으로써, 프론트엔드가 안정적으로 파싱할 수 있도록 설계.

11.9 응답 지연이 잦을 경우 고려할 개선 옵션

- 실제로 Gemini 무료 티어의 분당, 일일 쿼터에 도달해 응답 지연을 겪은 경험을 바탕으로 다음 개선 옵션을 고려
- 첫째, 이미 도입한 24시간 캐시의 유효 기간을 상황에 맞게 조정하거나, 인기 지역에 대해서는 사전 캐시를 미리 채워두는 방식.
- 둘째, 유료 플랜으로 전환하여 쿼터 자체를 확대.
- 셋째, 요청이 몰릴 때 더 가벼운 모델로 자동 전환하는 대체 로직 추가.
- 넷째, 프론트엔드에서 동일 조건 재요청을 일정 시간 내 막는 방식 적용.
- 다섯째, 결과를 스트리밍으로 받아 체감 대기시간을 줄이는 방식을 고려.

11.10 AI 기능을 2개로 늘린다면 프론트, 백엔드 확장 방안

현재 구조는 기능별로 이미 모듈이 분리되어 있어 확장이 용이함.

예를 들어 여행 일정표 자동 생성 AI 기능을 추가한다면, 백엔드에는 새로운 로직 모듈을 신설하고 라우트를 추가하며, 기존과 동일하게 커스텀 예외 클래스와 캐시 네임스페이스를 재사용.

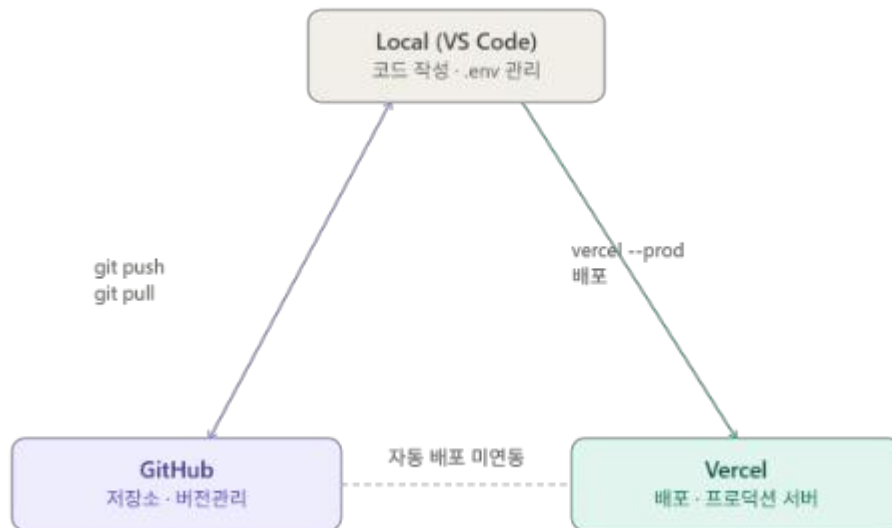
프론트엔드에는 기존과 동일한 조회, 렌더링 함수 쌍을 추가하고, tripState에 필드를 확장하며, 화면 구조에 새 섹션과 그 안의 로딩, 에러, 목록 컨테이너를 동일한 명명 규칙으로 추가.

즉, 라우트 1개, 모듈 1개, 조회와 렌더링 함수 쌍 1세트, 화면 섹션 1개라는 반복 가능한 패턴으로 기능을 확장할 수 있도록 설계되어 있음.

12. 사용 명령어 정리

구분	명령어	설명
1) 프로젝트 실행 폴더 생성 및 파일 생성	mkdir TripMate-AI	프로젝트 최상위 폴더 생성
	cd TripMate-AI	프로젝트 폴더로 이동
	mkdir css, mkdir js, mkdir api, mkdir images	하위 폴더 구조 생성 (VS Code 탐색기 New Folder로도 진행)
2) 셋업 (venv·라이브러리 설치·GitHub 연결)	node --version/ python --version	필수 실행환경 버전 확인
	python -m venv venv	Python 가상환경 생성
	.\venv\Scripts\Activate.ps1	가상환경 활성화
	Set-ExecutionPolicy -Scope CurrentUser - ExecutionPolicy RemoteSigned	venv 활성화 스크립트 실행 차단 오류 해결
	python -m pip install --upgrade pip	pip 최신화
	pip install google-gemini python-dotenv requests	Gemini API 등 핵심 패키지 설치
	pip install flask	로컬 테스트 서버용 Flask 설치
	npm install -g vercel	Vercel CLI 전역 설치
	git init	Git 저장소 초기화
	git add .gitignore→ git commit -m "..."	.gitignore최초 커밋 (보안 설정 선반영)
	git branch -M main	기본 브랜치명을 main으로 설정
	git remote add origin <저장소 URL>	GitHub 원격 저장소 연결
	git remote -v	원격 저장소 연결 상태 확인
3) 로컬 개발·테스트	python local_server.py	Flask 로컬 서버 실행 (127.0.0.1:5000)
	(브라우저) Ctrl+Shift+R	강력 새로고침으로 캐시 무시 재확인
	node --check js/app.js	JS 파일 문법 오류 사전 검증
4) Git 버전관리 (반복 작업)	git status	변경/스테이징 상태 확인 (.env노출 여부 매번 점검)
	git check-ignore -v .env	.env가 정상적으로 무시되는지 검증
	git add .	전체 변경사항 스테이징
	git commit -m "..."	커밋 (기능 단위로 다수 반복)
	git push	GitHub로 반영
	git log --oneline -5	최근 커밋 이력 확인 (문제 진단용)
	git branch -a	로컬/원격 브랜치 상태 확인
	git rebase --abort	꼬인 rebase 상태 안전하게 중단
5) Vercel 배포 및 운영	vercel login	Vercel 계정 인증 (세션 만료 시 재실행)
	vercel	최초 배포 (Preview 환경)
	vercel --prod	프로덕션 배포 (환경변수 반영, 기능 수정 후 매번 재배포)
	vercel logs <배포 URL>	실 서버리스 함수 요청/응답 로그 확인 (404·502 등 배포 오류 진단)

13. 구조



13.1 Local(VS Code) ↔ GitHub

- 1) 로컬에서 git init으로 저장소 초기화
- 2) git remote add origin <GitHub 저장소 URL>로 원격 저장소 연결
- 3) 이후 git add→ git commit→ git push로 로컬 변경사항을 GitHub에 업로드, 필요 시 git pull로 원격 변경사항을 로컬에 반영

13.2 Local(VS Code) → Vercel

- 1) npm install -g vercel로 CLI 설치, vercel login으로 계정 인증
- 2) 프로젝트 폴더에서 vercel실행 시 Vercel이 새 프로젝트를 생성하고 Preview 배포
- 3) vercel --prod실행 시 현재 로컬 코드 상태를 그대로 프로덕션에 배포

13.3 GitHub ↔ Vercel (이번 프로젝트에서는 미사용)

- Vercel 프로젝트 생성 시 GitHub 저장소를 연결하면, 이후 GitHub에 push할 때마다 Vercel이 자동으로 재배포하는 것이 표준 방식
- 이번 프로젝트에서는 저장소 연결 시도가 실패해서(Private 저장소 접근 권한 이슈), 로컬에서 vercel --prod로 직접 배포하는 방식을 사용
- 즉 GitHub은 "코드 백업·버전관리" 역할만, 실제 배포 트리거는 매번 로컬 터미널에서 수동으로 실행한 구조